Machine Learning Based Firewall: A Practical Machine Learning Application in Cyber Defence

Alejandro Panigada

Bsc Ethical Hacking, 2026

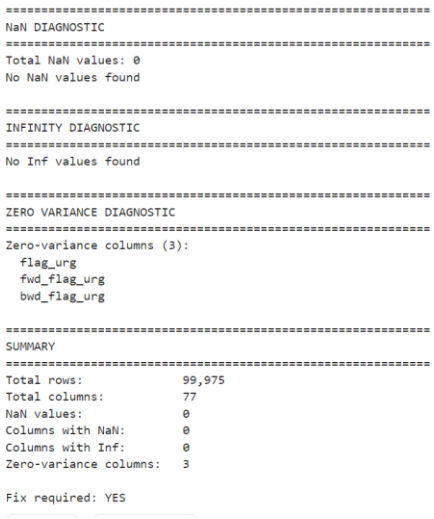
Faculty of Design, Informatics and Business
Abertay University

Table of Contents

Acknowledgements ix
Abstract x
Abbreviations, Symbols and Notation xii
Chapter 1 Introduction 1
1.1 Background Information 1
1.2 Aims and Objectives 3
1.3 Research Questions 4
1.4 Scope and Limitations 4
1.5 Research Contribution 5
1.6 Dissertation Structure 6
Chapter 2 Literature Review 7
2.1 Intrusion Detection Systems and Machine Learning 7
2.2 Feature Selection for High-Dimensional Network Data 8
2.3 Class Imbalance in Intrusion Detection 10
2.4 Deep Learning Approaches to Intrusion Detection 11
2.5 Evaluation Methodology for Imbalanced Classification 12
Chapter 3 Methodology 13
3.1 Dataset 13
3.2 Preprocessing 15
3.2.1 Data Quality Diagnostics 15
3.2.2 Label Encoding 15
3.2.3 Train/Test/Validation Split 15
3.2.4 Feature Scaling 16
3.3 Feature Selection 16
3.4 Model Selection and Development 18
3.4.1 XGBoost Baseline 18

3.4.2 Deep Learning model based on FT-Transformer 18
3.5 Model Evaluation 20
3.6 Model Reproducibility 20
Chapter 4 Results 21
4.1 Feature selection 22
4.2 XGBoost 23
4.2.1 Deep Learning model based on FT-Transformer 23
4.2.2 Per-class performance analysis 24
4.2.3 Problem Classes: Slowhttptest and FTP-Patator 24
4.3 Deep Learning 25
4.3.2 Per-class performance analysis 26
4.3.3 Comparative Analysis Summary 27
Chapter 5 Discussion 29
5.1 XGBoost and Deep Learning Comparison 29
5.2 Persistent misclassification of FTP-Patator and DoS Slowhttptest classes 31
5.2.1 Root Cause: Flow-Level Behavioural Similarity 31
5.2.2 Asymmetric Confusion Pattern and Feature Importance Analysis 32
5.2.3 Production Implications: Tiered Deployment Strategy 33
5.3 Implications for Autonomous Firewall Design 34
5.4 Limitations of the study 35
Chapter 6 Conclusion and Future Work 39
6.1 Summary of Research Questions and Results 39
6.2 Research Contributions 41
6.3 Future Work 42
List of References 44
Bibliography 50

Table of Figures



```
============================================================
NaN DIAGNOSTIC
============================================================
Total NaN values: 0
No NaN values found

============================================================
INFINITY DIAGNOSTIC
============================================================
No Inf values found

============================================================
ZERO VARIANCE DIAGNOSTIC
============================================================
Zero-variance columns (3):
  flag_urg
  fwd_flag_urg
  bwd_flag_urg

============================================================
SUMMARY
============================================================
Total rows:             99,975
Total columns:          77
NaN values:             0
Columns with NaN:       0
Columns with Inf:       0
Zero-variance columns:  3

Fix required: YES
```

Figure A-1 - Diagnostics 1

```
Non-numeric columns in X:
Count: 0
[]

Dtype summary:
float64    48
int64      29
Name: count, dtype: int64

No non-numeric columns found — no encoding needed
```

Figure A-2 – Diagnostics 2

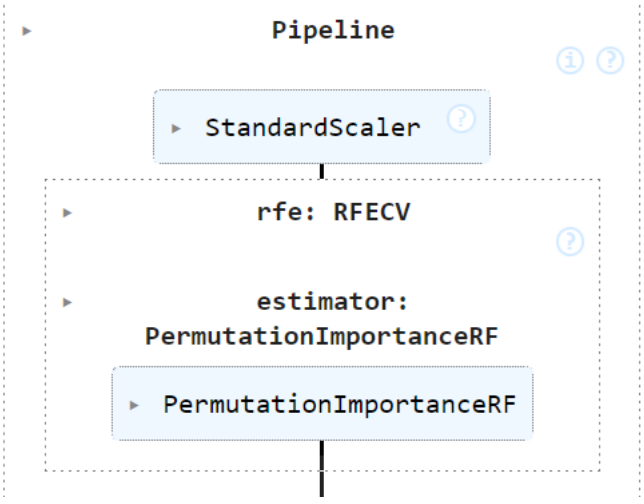


Figure A-3 – Feature selection pipeline

```
Features selected: 56
Selected features: ['dst_port', 'fwd_pkt_per_s', 'bwd_pkt_per_s', 'fwd_pkt_hdr_len_min', 'bwd_pkt_cnt', 'bwd_pkt_len_mean', 'bwd_pkt_len_std', 'bwd_pkt_hdr_len_tot', 'bwd_pkt_hdr_len_min', 'bwd_non_empty_pkt_cnt', 'iat_max', 'iat_min', 'iat_mean', 'iat_std',
'fwd_iat_tot', 'fwd_iat_max', 'fwd_iat_min', 'fwd_iat_mean', 'fwd_iat_std', 'bwd_iat_tot', 'bwd_iat_max', 'bwd_iat_min', 'bwd_iat_mean', 'bwd_iat_std', 'active_max', 'active_min', 'active_mean', 'active_std', 'idle_max', 'idle_min', 'idle_mean', 'idle_std',
'flag_SYN', 'flag_fin', 'flag_rst', 'flag_ack', 'flag_psh', 'fwd_flag_psh', 'bwd_flag_psh', 'flag_urg', 'fwd_flag_urg', 'bwd_flag_urg', 'flag_cwr', 'flag_ece', 'fwd_bulk_bytes_mean', 'fwd_bulk_pkt_mean', 'fwd_bulk_rate_mean', 'bwd_bulk_bytes_mean', 'bwd_bulk
_pkt_mean', 'bwd_bulk_rate_mean', 'fwd_subflow_bytes_mean', 'fwd_subflow_pkt_mean', 'bwd_subflow_bytes_mean', 'bwd_subflow_pkt_mean', 'fwd_tcp_init_win_bytes', 'bwd_tcp_init_win_bytes']

Shapes:
  Train: (69982, 56)
  Val:   (14996, 56)
  Test:  (14997, 56)

DataFrames created
  X_train_df: (69982, 56)
  X_val_df:   (14996, 56)
  X_test_df:  (14997, 56)
```

Figure A-4 – Feature Selection Results

```
[0]     validation_0-mlogloss:2.06534   validation_1-mlogloss:2.06522
[50]    validation_0-mlogloss:0.14065   validation_1-mlogloss:0.13958
[100]   validation_0-mlogloss:0.02975   validation_1-mlogloss:0.02853
[150]   validation_0-mlogloss:0.01996   validation_1-mlogloss:0.01923
[200]   validation_0-mlogloss:0.01823   validation_1-mlogloss:0.01835
[233]   validation_0-mlogloss:0.01766   validation_1-mlogloss:0.01836
Final Train mlogloss: 0.0177
Final Val  mlogloss: 0.0184
```

Figure A-5 – Training and Validation loss

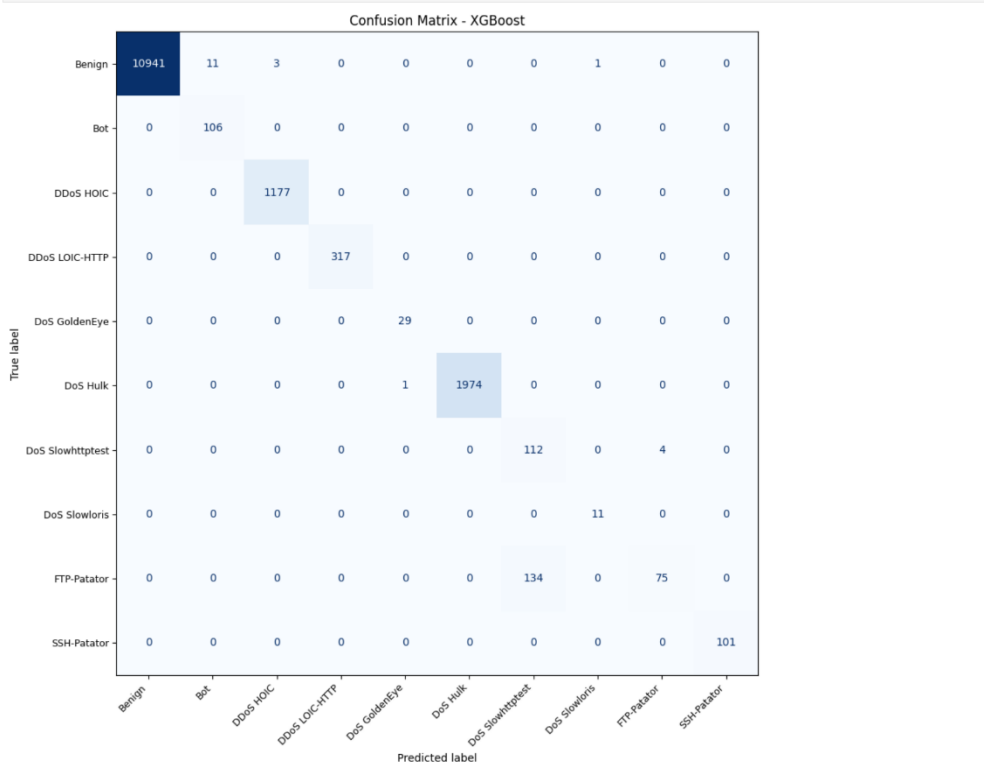


Figure A-6 – XGBoost Confusion Matrix

	precision	recall	f1-score	support
Benign	1.0000	0.9986	0.9993	10956
Bot	0.9060	1.0000	0.9507	106
DDoS HOIC	0.9975	1.0000	0.9987	1177
DDoS LOIC-HTTP	1.0000	1.0000	1.0000	317
DoS GoldenEye	0.9667	1.0000	0.9831	29
DoS Hulk	1.0000	0.9995	0.9997	1975
DoS Slowhttptest	0.4553	0.9655	0.6188	116
DoS Slowloris	0.9167	1.0000	0.9565	11
FTP-Patator	0.9494	0.3589	0.5208	209
SSH-Patator	1.0000	1.0000	1.0000	101
accuracy			0.9897	14997
macro avg	0.9191	0.9322	0.9028	14997
weighted avg	0.9941	0.9897	0.9893	14997

Figure A-7- XGBoost Classification Report

```
Device: cuda
Features: 56, Classes: 10
Train batches: 274 | Val batches: 59 | Test batches: 59
Output shape: torch.Size([256, 10])
Trainable parameters: 281,930
Class weights: tensor([  0.1369,  14.2530,   1.2740,   4.7317,  51.0818,   0.7592,  12.9837, 132.0415,   7.1998,  14.7953], device='cuda:0')
Epoch 01/50 | Train Loss: 0.3156 F1: 0.6251 | Val Loss: 0.1525 F1: 0.8706 AUC: 0.9995
Epoch 02/50 | Train Loss: 0.1755 F1: 0.8188 | Val Loss: 0.1612 F1: 0.8433 AUC: 0.9996
Epoch 03/50 | Train Loss: 0.1533 F1: 0.8294 | Val Loss: 0.1276 F1: 0.8425 AUC: 0.9998
Epoch 04/50 | Train Loss: 0.1450 F1: 0.8434 | Val Loss: 0.1339 F1: 0.8897 AUC: 0.9997
Epoch 05/50 | Train Loss: 0.1580 F1: 0.8354 | Val Loss: 0.1353 F1: 0.8896 AUC: 0.9998
Epoch 06/50 | Train Loss: 0.1536 F1: 0.8526 | Val Loss: 0.1235 F1: 0.8856 AUC: 0.9998
Epoch 07/50 | Train Loss: 0.1431 F1: 0.8484 | Val Loss: 0.1274 F1: 0.8658 AUC: 0.9997
Epoch 08/50 | Train Loss: 0.1331 F1: 0.8687 | Val Loss: 0.1673 F1: 0.8612 AUC: 0.9996
Epoch 09/50 | Train Loss: 0.1321 F1: 0.8755 | Val Loss: 0.1488 F1: 0.8694 AUC: 0.9997
Epoch 10/50 | Train Loss: 0.1335 F1: 0.8715 | Val Loss: 0.1322 F1: 0.8857 AUC: 0.9997
Epoch 11/50 | Train Loss: 0.1274 F1: 0.8857 | Val Loss: 0.1315 F1: 0.8594 AUC: 0.9997
Epoch 12/50 | Train Loss: 0.1272 F1: 0.8842 | Val Loss: 0.1188 F1: 0.8867 AUC: 0.9998
Epoch 13/50 | Train Loss: 0.1281 F1: 0.8815 | Val Loss: 0.1226 F1: 0.8846 AUC: 0.9997
Epoch 14/50 | Train Loss: 0.1271 F1: 0.8777 | Val Loss: 0.2611 F1: 0.8691 AUC: 0.9997
Early stopping at epoch 14

Best Val F1 Macro: 0.8897

Evaluating on test set...
Test F1 Macro: 0.8843
```

Figure A-8 – Training Loop with Training Loss and Validation Loss

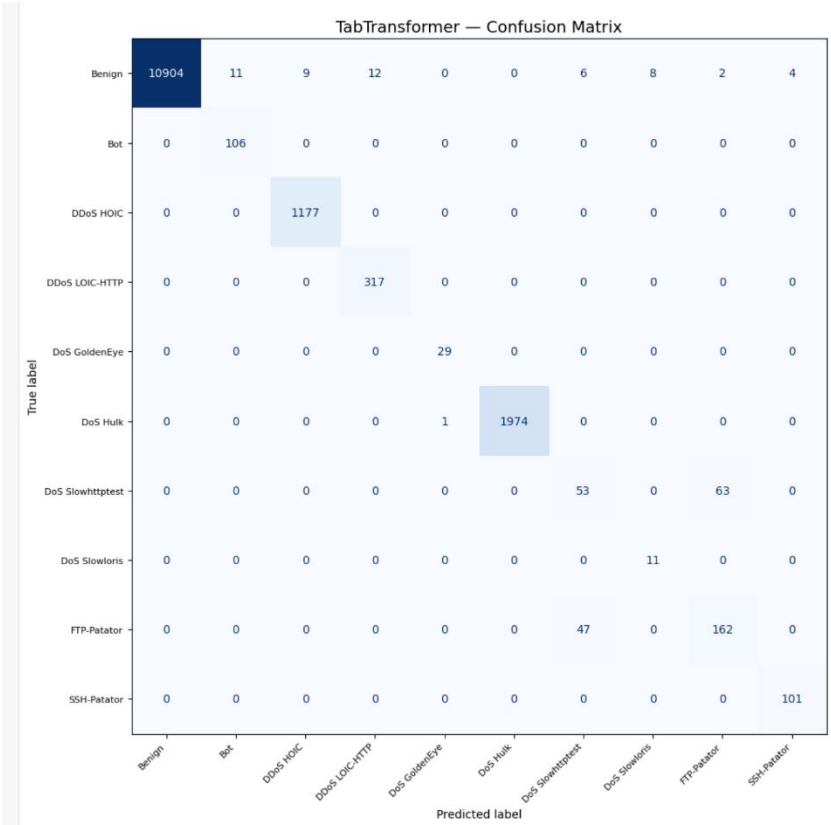


Figure A-9 – Transformer-based Confusion Matrix

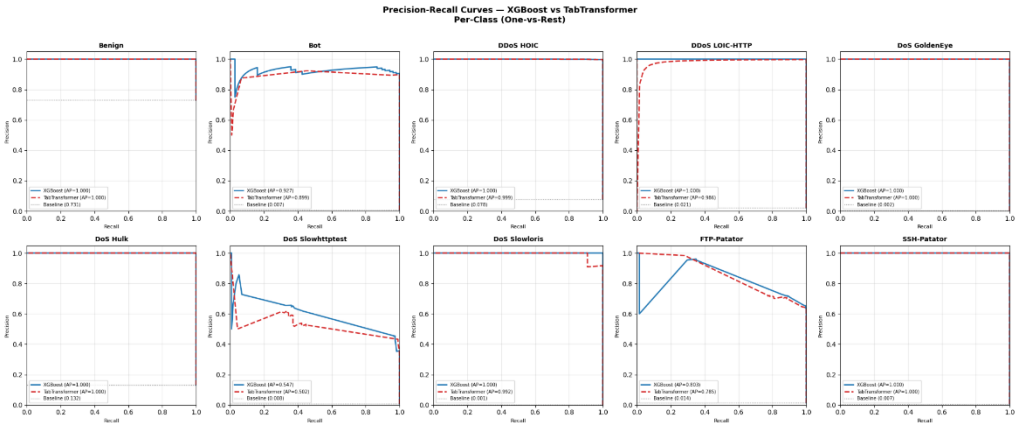


Figure A-10 – Precision-Recall Curves

```
FINAL COMPARISON
```

xgbModel	F1 Macro	F1 Weighted	MCC	AUC Macro
XGBoost	0.9028	0.9893	0.9770	0.9994
TabTransformer	0.8954	0.9907	0.9800	0.9993

xgbModel	FTP-Patator F1	DoS Slowhttp F1
XGBoost	0.5208	0.6188
TabTransformer	0.7768	0.4324

Figure A-11 – Final Comparison of XGBoost and Transformer-based on problematic classes

```
TabTransformer
F1 Macro   : 0.8954  ← primary metric
F1 Weighted: 0.9907
MCC        : 0.9800
AUC Macro  : 0.9993
```

	precision	recall	f1-score	support
Benign	1.0000	0.9974	0.9987	10956
Bot	0.8833	1.0000	0.9381	106
DDoS HOIC	0.9975	1.0000	0.9987	1177
DDoS LOIC-HTTP	0.9906	1.0000	0.9953	317
DoS GoldenEye	0.9667	1.0000	0.9831	29
DoS Hulk	1.0000	0.9995	0.9997	1975
DoS Slowhttptest	0.5797	0.3448	0.4324	116
DoS Slowloris	0.7333	1.0000	0.8462	11
FTP-Patator	0.7043	0.8660	0.7768	209
SSH-Patator	0.9712	1.0000	0.9854	101
accuracy			0.9911	14997
macro avg	0.8827	0.9208	0.8954	14997
weighted avg	0.9910	0.9911	0.9907	14997

Figure A-12 – Final Transformer-Based classification report

```
============================================================
  XGBoost
============================================================
  F1 Macro   : 0.9028  ← primary metric
  F1 Weighted: 0.9893
  MCC        : 0.9770
  AUC Macro  : 0.9994

                precision    recall  f1-score   support

        Benign     1.0000    0.9986    0.9993     10956
           Bot     0.9060    1.0000    0.9507       106
      DDoS HOIC     0.9975    1.0000    0.9987      1177
  DDoS LOIC-HTTP     1.0000    1.0000    1.0000       317
   DoS GoldenEye     0.9667    1.0000    0.9831        29
        DoS Hulk     1.0000    0.9995    0.9997      1975
DoS Slowhttptest     0.4553    0.9655    0.6188       116
   DoS Slowloris     0.9167    1.0000    0.9565        11
     FTP-Patator     0.9494    0.3589    0.5208       209
     SSH-Patator     1.0000    1.0000    1.0000       101

        accuracy                         0.9897     14997
       macro avg     0.9191    0.9322    0.9028     14997
    weighted avg     0.9941    0.9897    0.9893     14997
```

Figure A-13 – Final XGBoost classification report

Table of Tables

Acknowledgements

Thanks to my family, which supported me all the way and still does so all the time.
Thanks to Romina, Guglielmino, Michele, Lavinia, Zaira, Melania, Bosia and all the people who helped making my graduation possible.

Without their help, I would not have been able to accomplish it.

Thanks to Sanaz for the support in supervising my work.

Abstract

Network Intrusion Detection Systems(NIDS) are a critical component to modern cybersecurity.
Modern attacks are hard to detect so administrators rely on such technology to perform autonomous detection to halt them before they can perform permanent damage to the organization.

Usage of such technologies in production has several challenges, including high class imbalance in datasets used for training, and difference of performance across the different datasets, as reported in literature, which might carry in production environments making the systems potentially unusable.

This dissertation compares XGBoost and a transformer-based model based on TabTransformer and FT-Transformer. The dataset used, LyCoS-Unicas-2018 was sampled using stratified sampling, for maintaining class proportions, and this produced a reduced subset of 100,000 instances.
Macro F1 was the main metric. Aggregate accuracy is considered misleading on highly imbalanced data so it was only reported across the study for reference.

The two models share same dataset, preprocessing, train/validation/split process and class weight scheme. This is done to have a fair comparison between the two architectures.

Feature selection was performed using RFECV with a premutation-based feature selection chosen for its ability to retain minority classes useful for detection. RFECV produced a 56-feature vector used in both models.

XGboost achieved an F1 score of 90.28% and the transformer-based model 89.54%. Both models failed consistently on 2 of the 10 classes with no cross-contamination on the others. This suggests a flow-level

feature limitation in the LycosStand feature extraction process rather than being a model architecture problem. XGBoost obtained a higher result than the Transformer-based model. However, the literature suggests that the two model performance might be less significant on larger datasets.

A tiered deployment strategy is suggested where the 8 high-confidence classes are autonomously handled while the remaining two problematic classes are sent to be analysed to human analysts.
A large volume of false positives is possible so testing is required before putting the system into production.

Last consideration is that traffic evolves and new attacks alter the input patterns. To avoid unexpected degrade in performance and misclassification, administrators must test regularly the system for degradation and have to retrain the model with new data.

Abbreviations, Symbols and Notation

ML - Machine Learning

DL - Deep Learning

NIDS - Network Intrusion Detection System

IDS - Intrusion Detection System

RFECV - Recursive Feature Elimination with Cross-Validation

RFE - Recursive Feature Elimination

RF - Random Forest

MCC - Matthews Correlation Coefficient

AUC - Area Under the Curve

PRC - Precision-Recall Curve

FPR - False Positive Rate

TPR - True Positive Rate

SMOTE - Synthetic Minority Over-sampling Technique

LSTM - Long Short-Term Memory

RNN - Recurrent Neural Network

NLP - Natural Language Processing

DPI - Deep Packet Inspection

PCAP - Packet Capture

TCP - Transmission Control Protocol

UDP - User Datagram Protocol

HTTP - Hypertext Transfer Protocol

FTP - File Transfer Protocol

SSH - Secure Shell

DoS - Denial of Service

DDoS - Distributed Denial of Service

GPU - Graphics Processing Unit

CPU - Central Processing Unit

SHAP - SHapley Additive exPlanations

Chapter 1 Introduction

1.1 Background Information

The continuous proliferation of network systems and the increasing complexity of cyber threats makes network intrusion detection a critical component of modern cybersecurity. Organisations across several sectors continue to face an increasing number and diversity of attacks such as polymorphic malware, based on code that changes itself to avoid signature based detection or agentic AI attacks where AI agents scan, identify, and exploit systems autonomously and faster than humans (Catalano *et al.*, 2022; Fang *et al.*, 2024).

Although there are various types of Intrusion Detection Systems (IDSs), each type has its own strengths and weaknesses. Traditional signature-based IDSs are effective against known attack signatures, however, they are vulnerable against variants or zero-day exploits where a signature is non-existent(Wang,2004). Anomaly-based intrusion detection instead offers a complementary approach based on statistical representations of normal network behaviour and flags deviations as potential offences targeting the organisation (Wang,2004).

Machine Learning (ML) algorithms have emerged as the dominant paradigm for anomaly-based detection by being able to generalise across attack variants with no necessity of manual signature creation. The deployment of ML in Network Intrusion Detection System (NIDS) has its own difficulties. Apart from classification performance, challenges can be severe class imbalance in the dataset or computational cost for real-time inference (Ali *et al.*, 2024; Zhang, Muniyandi and Qamar, 2025).

The main challenge lies in making ML-based solution usable in a production environment. Published ML IDS report high accuracy in classification task at around 99%, but these figures often are artefacts of class imbalance, where benign traffic compromises most of the network flows, obscuring real detection capabilities. The present study addresses

this by using more appropriate metrics, such as the F1 score rather than accuracy. This makes results more accurate regarding the capabilities of detecting attacks across all attacks.

Aggregate accuracy fails when dealing with imbalanced datasets because it consists on averaging performance across classes weighting by their frequency.

Benign traffic includes 73% of all traffic and using aggregate accuracy with a model that guesses only benign class would achieve a 73% of accuracy without predicting any of the attacks.

Per-class evaluation reveals reports instead performance for each attack class individually being more informative in a security context where detection of all classes accurately is more important than aggregate accuracy.

Also, specific failures that are invisible at the aggregate levels are visible when using per-class evaluation.

As it has been the case with this research, even with a high F1 score, specific classes might still be misclassified and by using aggregate metrics, the misclassified classes cannot be detected.

Because of these reasons per-class precision and per-class recall alongside macro-averaged F1 are the main metric used for this research. These metrics were used not only for the models themselves but for feature selection as well.

1.2 Aims and Objectives

The research study aims to investigate the application and compare the performance of both classic ML algorithms, such as XGBoost and Deep Learning, on NIDS traffic. The research objectives are as follows.

- Build a reliable preprocessing pipeline for the LyCos-Unicas-2018 dataset
- Implement and evaluate the XGBoost algorithm towards the pre-processed data
- Implement a Transformer-based Deep Learning algorithm on the pre-processed data and compare the results with the ones obtained from XGBoost
- Identify problems with implementation and data processing
- Propose fixes for any problems found during testing

The comparison between the Transformer-based model and XGBoost is structured to maintain all variables fixed apart model themselves.
This means having same datasets, preprocessing, and using the same training, validation, and test data.

Class imbalance is addressed by using class weights computed in the same way.

Hyperparamerter tuning was performed separately for both models as the they have different sets of them, and this makes impossible to directly have a comparison.

The configurations used typical defaults, rather than performing exhaustive research. This is because the research is based on analysing general performance of the models and related methodology, rather than achieving their individual maximum performance.

This means in practice that the figures reported for both models are used to represent a reproducible setup rather than one fully optimized.

Also, reproducibility ensures interpretability of results allowing a more accurate comparison between the two models.

While the relative figures are meaningful for comparison, a more aggressive approach in hyperparameter tuning, especially for the Transformer-based model, might improve performance and possibly change the actual ranking.

1.3 Research Questions

The defined research questions are as follows.
RQ1: Can ML and Deep Learning be used effectively in the NIDS domain?
RQ2: What are the performances of ML compared to Deep Learning?
RQ3: Can both ML and Deep Learning be used effectively in a production-ready environment?

1.4 Scope and Limitations

The research study is limited to the usage of the LyCoS-Unicas-2018 dataset. Because of computational constraints, only a stratified sample of 100,000 records was selected from the entire dataset.
A stratified sample was decided because it allows to maintain relative proportions of each class. This also means that classes still are underrepresented in the dataset

Models are evaluated in an offline setting. Proof of concept is limited to automated rule generation from model output. The proof of concept is not a production-ready deployment.

Also, evaluating the models offline, rather than on real world production environment, makes out of scope several aspects.

First, real-time latency in inference is not benchmarked under real network load. Since all the features are to be extracted from raw traffic for

usage by the model, there is high risk of degraded performance during deployment in real-time network analysis.

Also the gradual change of signatures because of attack evolution over time, the main reason to use anomaly-based detection, was not tested because it was used a single dataset for evaluation.

It was not tested adversarial robustness either which consists in analysis of traffic carefully shaped to avoid detection.

Lastly, cross-data generalisation, where a model is trained on one dataset and tested on another one, was not evaluated.

These omissions are deliberate scope choices to ensure the project was achievable in both timing and costs.

Additionally, this work main scope is answering the previously introduced research questions, not trying to solve the highly complex problems that persist in both academic and industrial sector regarding the application of ML and DL based IDS.

Some of these problems are presented in Chapter 6 in the form of future work suggestions.

1.5 Research Contribution

This research study makes three fundamental contributions to the anomaly-based NIDS field.

First, it provides a systematic performance comparison on tabular data between ML algorithm XGBoost and a based transformer-based DL

algorithm. While most of the existing literature showcases individual algorithm effectiveness, direct comparison and usage of more adequate metrics such as F1 score remains limited.

Secondly, the research study uses a permutation-based feature importance instead of the standard impurity-based selection. This is done to avoid bias towards the majority class. Both types of feature importance were used and compared using model performance as a metric.

Third, the research addresses the feasibility of production deployment through analysis of the obtained performance. Aggregate accuracy is not enough for defining this. Using metrics such as per-class evaluation, it is possible to find out model reliability and when the model needs additional handling, as is the case for the Slowhttptest and FTP-Patator classes, where detection percentage is unacceptable for production environments, due to an extremely high percentage of misclassification.

1.6 Dissertation Structure

Chapter 2 covers a review of relevant literature involving the usage of ML for NIDS, methodology for feature selection, class imbalance handling, and the usage of DL algorithms for tabular data.
Chapter 3 explains the methodology used. This includes data preparation and preprocessing, model choice and implementation, and evaluation of results.
Chapter 4 presents all the results obtained from experimentation.
Chapter 5 presents a discussion about the meaning of the obtained results.
Chapter 6 concludes the research study and highlights the possible future work that can be conducted and concludes the study.

Chapter 2 Literature Review

This chapter provides the required background information to provide context for the chosen methodology explained in Chapter 3.
A general history and evolution regarding the usage of ML in intrusion detection is provided. Then, the feature selection techniques for the NIDS dataset are discussed, followed by the imbalance handling found in IDS datasets. It also covers the usage of deep learning on Network Intrusion Detection and proper evaluation metrics and methodologies in imbalanced datasets.

2.1 Intrusion Detection Systems and Machine Learning

For more than two decades, ML applications in network intrusion detection have been an active area of research. Traditionally, IDS have been implemented using a signature-based detection method. It focuses on using predefined rule sets for the correct identification of attack patterns. Signature-based detection systems work with enough accuracy towards known attack patterns, but are incapable of recognising unseen samples, making it unsuitable for zero-day or novel threats (Wang, 2004). Limitations of signature-based detection have brought motivation to find alternatives, focusing on anomaly-based detection using ML and Deep Learning (DL) algorithms (Khraisa et. Al, 2019).
Early ML algorithms used were shallow classifiers, such as Decision Trees or Support Vector Machines. Those shallow classifiers proved to be effective on balanced datasets, but not on imbalanced data, which is usually the case for network traffic analysis, where benign traffic is found to be exponentially more represented than the malicious traffic classes (Chen et al., 2024). This problem was addressed better after the emergence of ensemble methods, such as Random Forest (RF)(Breiman, 2001), and Gradient Boosting frameworks such as XGBoost (Chen and Guestrin, 2016). These classes of algorithms allowed superior generalisation and much more robustness to class imbalance.

Even with the emergence of ensemble methods, network anomaly detection has structural difficulties that no algorithm can fully address.

Sommer and Paxson (2010) identified several of them.
First, the diversity of network traffic is extremely high. Even in a single network, the most basic characteristics such as bandwidth, connection duration, or application mix can have an immense variety making the detection of attacks in such a short interval of time unfeasible.
Also, anomaly-based detection systems, are based on finding abnormalities among statistically normal traffic. So much variation in the input data makes difficult for anomaly-based systems to establish a baseline of what is normality. Secondly, packet variety does not extend only to packet features at the network level. It also has enormous variance in the application-level.
Semantically, application-level characteristics are at least as much fluctuating as packet-level ones. Lastly, traffic properties tends to variate depending on the time interval selected for reference. Long intervals of time make characteristics more stable(hours, days, weeks).
For example, as stated by Sommer and Paxson (2010), if during lunchtime an abnormal behaviour in the network occurs, it is easier to find it as prior weeks have a reliable history of normal traffic during that time of the day.

2.2 Feature Selection for High-Dimensional Network Data

Datasets consisting of network data have high dimensionality as a principal characteristic. The feature number is usually around 70-80. Having so many features for each record brings substantial noise into the classification algorithms, increases the computation cost and through the

curse of dimensionality, degrades model performance (Agrawal, 2025). This makes feature selection critical for NIDS based on those algorithms.

Recursive Feature Elimination (RFE), introduced by Guyon et al. (2002), iteratively removes the least important features based on a base estimator's importance score. If combined with cross-validation (RFECV), a chosen subset of features which maximises a chosen performance metric is selected, while also allowing for checking for overfitting (Awad and Fraihat, 2023). However, it appears that impurity-based importance measures, found commonly as default in RF, exhibit bias toward high-cardinality features, distorting the outcome of the selection (Strobl et al., 2007).

To address this, using permutation-based importance measures or SHAP (SHapley Additive exPlanations) values can be preferred instead (Janitza et al., 2013; Strobl et al., 2007; Lundberg, 2017). Also, it is important to consider which is the most suitable optimisation metric. When interacting with imbalanced data, especially common in network traffic data, optimising for accuracy or weighted F1 when performing RFE, will allow for the retention of features that can discriminate towards the majority class. Instead, selecting features based on micro-averaged F1 will force the selection to choose features that improve the performance equally across all the classes (Abdelaziz,2025). This is helpful, especially to represent rare categories across the dataset, which would otherwise be deprioritised (Leevy et al., 2018).

SHAP (Lundberg, 2017) was considered too for the feature selection phase. SHAP values produce a contribution score foreach feature based game theory making them valuable for interpretation.

The problem is that SHAP is computationally costly compared to permutation importance even more when used inside a cross-validation loop like RFECV. Permutation importance was the final selection and SHAP was noted as possible suggestion for future work.

The base estimator for RFE is also important. Random Forest is one of the most common choices because of its stability and speed under cross-validation but its default impurity-based importance as mentioned earlier carries bias.
The estimator does not have to use the same importance measure as the selection wrapper so in this study Random Forest was used as base estimator and permutation importance is computed on top of it. This allows Random Forest to operate at maximum speed while avoid inheritance of the bias present in its default impurity-based ranking.

2.3 Class Imbalance in Intrusion Detection

When dealing with network intrusion detection, class imbalance is a common challenge. Attack traffic commonly represents a very small percentage of the total incoming and outgoing flow (Goldschmidt & Chuda, 2025). Leevy et al. (2018), provide a comprehensive review of several techniques to deal with high-class imbalance in large datasets. Approaches used are categorised in resampling, cost-sensitive learning, and hybrids between these last two methods.

A common resampling technique is the Synthetic Minority Oversampling Technique (SMOTE), introduced by Chawla et al. (2002). Widely used, it consists of generating minority class samples by interpolating existing instances in feature space. SMOTE, however, can be ineffective when the majority classes exhibit feature overlap between majority and minority classes (Blagus & Lusa, 2013). This is important when dealing with IDS, as certain attack categories generate flows very similar to benign traffic.

Cost-sensitive approaches, such as assigning class-weighted loss penalties or sample weights inversely proportional to class frequency, can be an effective method to avoid distributional assumptions found in SMOTE.

The choice between the usage of resampling and cost-sensitive learning highly depends on the dataset. SMOTE has higher performance when the minority class is represented sufficiently but small. If the minority class contains errors, the synthetic samples will also inherit those errors making it less ideal in those scenarios.
Cost-sensitive learning instead avoids generating synthetic samples and because of that is more robust. For this reason,this study uses cost-sensitive learning in both feature selection and model training rather than SMOTE.

2.4 Deep Learning Approaches to Intrusion Detection

Deep learning (DL) has been used extensively in several areas, such as computer vision and Natural Language Processing (NLP). The vast success obtained by using DL techniques in those areas motivated the use in other domains such as NIDS. An example is the usage of recurrent architectures such as Long Short-Term Memory (LSTM) to create a model able to detect malicious traffic based on the temporal dynamics of network flows. This case, however, assumes that the temporal flow of the inspected packets is preserved. This assumption does not hold in many cases, for example, when data is randomly sampled.

The advent of the Transformer architecture, created by Vaswani et al. (2017), addresses shortcomings of Recurrent Neural Networks (RNN) such as sequential processing limitations by using what is called self-attention. Self-attention is an attention mechanism. An attention mechanism is a ML technique which allow DL models to give priority to the most important and meaningful parts of input data.

Huang et al. (2020) adapted the Transformer architecture to tabular data, creating TabTransformer, capable of capturing interactions between features which would not otherwise be able to be tracked by conventional RNNs. However, even with the theory of using attention-based models for tabular data being appealing, Works, such as Grinsztajn et al. (2022), proved that tree-based models, especially Gradient Boosting frameworks, outperform DL methods in medium-sized datasets.

2.5 Evaluation Methodology for Imbalanced Classification

Selecting the most appropriate evaluation metrics is of critical importance when conducting IDS research. Network traffic datasets are often highly imbalanced, with benign traffic being overly represented, making accuracy as a metric misleading when used as a performance indicator.

If a classifier labels all traffic as benign, then it will achieve an accuracy score equal to the proportion of benign traffic found in the dataset, which in the case of the Lycos-IDS2018 dataset amounts to approximately 73%, even when no attack is successfully detected. This means that accuracy scores will be high even if the model just classifies everything as benign.

Using the macro-averaged F1 score is, in this case, more accurate as a metric. Consisting of the unweighted mean of per-class F1 scores, it represents every attack class much more equally. Chicco & Jurman (2020), also proposed the use of the Matthews Correlation Coefficient(MCC) as the most appropriate single scalar metric when using imbalanced classification, since it accounts simultaneously for true positives and negatives, and false positives and negatives across all classes without having a ruling majority class, which alters the results.

Saito & Rehmsmeier (2015) suggested using as a minimum standard for IDS classifiers, per-class F1 Scores, the area under the Precision-Recall

Curve (PRC) and false positive rate. A high false positive rate is crucial as it makes an IDS unusable in practice.

The dataset used for the present research, LyCoS-Unicas-IDS2018 was introduced as a corrected version of the CSE-CIC-IDS2018 by Cantone et al. (2024). Several inconsistencies and artefacts were removed. However, the dataset is still highly imbalanced, so the previously mentioned techniques were used across the research.

Chapter 3 Methodology

This chapter explains the experimental procedure carried out during the research starting from the acquisition of the dataset, its preprocessing, model training and final evaluation. All the work conducted was performed on the platform Kaggle in a notebook environment using a P100 GPU accelerator. The experimentation pipeline was designed to avoid common pitfalls found in IDS literature, such as data leakage and overfitting.

3.1 Dataset

The dataset used for the experiments is LyCoS-IDS2018 (Cantone,2024). It is a corrected derivation of the CSC-CIC-IDS2018 (Sharafaldin et. Al 2018) dataset, addressing several labelling inconsistencies and several flow generation artefacts present in the original. The dataset includes 13,691,268 records subdivided into either benign traffic or one of the following 10 attack categories: Bot, DDoS HOIC, DDoS LOIC-HTTP, DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS Slowloris, FTP-Patator, and SSH-Patator.

Each record is described by 77 features, which were extracted from the raw CSE-CIC-IDS2018 PCAP file using LycoSTand (Rosay et. Al, 2021). LycosStand is a tool which forms network flows and extracts features characterising them.

Due to the constraints of both memory and computational resources, a stratified random sample of 100,000 records was extracted from the dataset. The usage of stratified sampling preserves the proportional class distribution found in imbalanced datasets, such as the one used in this study.

Rare classes with very few samples were removed entirely (DDoS LOIC-UDP, Web Attack - Brute Force, Web Attack - Sql Injection, Web Attack - XSS), bringing the number of classes from 14 to 10. The minimum number of samples required for a class to be retained is 35. Eliminating poorly represented classes is a drastic preprocessing strategy for handling class imbalance. In general, removing classes is not suggested as the model ends up losing information which could be useful for improving predictions. It also loses the ability to generalise those rare classes entirely.
However, in this scenario, classes with fewer than 35 samples following stratified sampling were removed prior to model training, as these classes would provide insufficient representation for StratifiedKFold cross-validation with k=5 folds during the RFECV procedure.

It is also important to note that LycosStand extract only flow-level features leaving aside packet-level characteristics such as packet-content or protocol-specific features.

This limits the classification as those aspects could be used to improve performance and separate problematic classes.

This also means the limitation extends to any model using this pipeline. Just using another model will not be able to fix the lack of information which is present in the data itself.
This can only be addressed at the feature extraction step which is considered out of scope for this project.
Any team using this work as reference should not worry primarily on the models used but rather in feature representation and feature extraction.

3.2 Preprocessing

3.2.1 Data Quality Diagnostics

Prior to any modelling, diagnostic was conducted for any NaN values, infinite values and zero-variance columns. Three zero variance columns were identified: flag_urg,fwd_flag_urg and bwd_flag_urg. They were not dropped as it was known they would be automatically ignored by the feature selection later in the experimentation.

No columns had any NaN or infinite values. All features were numeric so no encoding was needed except for the labels.

3.2.2 Label Encoding

Attack category labels were integer-encoded using LabelEncoder from the scikit-learn library, which was fitted on the sampled version of the dataset prior to the splitting operation.

3.2.3 Train/Test/Validation Split

After performing stratified random sampling on the full dataset and performing encoding on the labels, the resulting data was partitioned into three subsets: Training(70%), Validation(15%) and Test (15%) as shown in Figure B-1.

```
============================================================
DATASET SPLIT SUMMARY
============================================================
Split           Samples   Percentage
------------------------------------------------------------
Train            69,982       70.00%
Validation       14,996       15.00%
Test             14,997       15.00%
------------------------------------------------------------
Total            99,975      100.00%
============================================================
```

Figure B-1

The method was used with a fixed random seed of 42 (to ensure reproducibility), and the stratify option was enabled to ensure the ratio of the different classes would be the same among test, training and validation data.

3.2.4 Feature Scaling

Numerical features were standardised using StandardScaler class from scikit-learn library, fitted entirely on the training data and then applied to both training, test and validation data. Fitting the scaler on the full dataset before having it split into the several parts, would constitute a data leakage since test set statistics would influence scaling parameters (Pinheiro et al., 2025).

3.3 Feature Selection

Feature selection was performed first by using RFE with Cross-Validation (RFECV) using Random Forest as estimator. This method, by default, uses impurity-based feature selection. This can be problematic as impurity-based feature selection has a bias towards the majority class and toward high-cardinality features, which is unideal for an imbalanced dataset such as the one used in this research Strobl et al. (2007). Because of this, it was later switched to a permutation-based implementation by using a custom wrapper based on Strobl et al. (2007) and Janitza et al. (2013) work.

RFECV was configured with a StratifiedKFold cross-validation scheme (k=5, shuffled) and a step size of 10% remaining features per iteration with an arbitrary minimum feature count of 20.

The optimisation metric was set to macro-averaged F1 score rather than set to accuracy or weighted F1. This was chosen because of the class imbalance characteristic of the dataset chosen. If accuracy or weighted F1 were used, it would mean retaining features that discriminate the majority class (Benign), while the usage of F1 scoring allows equal weight across all attack categories, including rare and difficult to detect classes (Riyanto *et al.*, 2023).

Out of 77 features, RFECV extracted 35 features when using impurity-based and 56 features when using permutation-based.

Model final performance using impurity-based detection was not tested.

SHAP values were considered for usage in feature selection.
While SHAP values are theoretically feasible, and in many instances preferred, due to better interpretability and its per-instance attributions.

The reason for why permutation-based feature selection was the final one is because SHAP has high computational costs.
Especially due to usage of cross-validation, using SHAP would have meant a very slow feature selection step with no substantial advantage over permutation-importance.

Also, while SHAP provide better interpretability, it also is more cantered on providing reasons for why a model made specific predictions rather than ranking features for their contributions.
However, the interpretability of SHAP might be still useful for model performance analysis rather than for feature selection.

This is marked as future work in Chapter 6.

3.4 Model Selection and Development

3.4.1 XGBoost Baseline

The primary implemented classifier was XGBoost. The choice was based on the established superiority performance-wise on medium-scale tabular datasets (Grinsztajn, Oyallon and Varoquaux, 2022). The model was configured with 500 estimators, a learning rate of 0.05, a maximum tree depth of 6, a subsample ratio of 0.8 and a column subsample of 0.8.

A histogram-based tree was chosen for computational efficiency. It also helps to reduce memory and accelerate training without losing a significant amount of accuracy.

Sampling weights allow addressing class imbalance by weighting samples inversely proportional to the class frequency. This ensures every class, including rare classes receives adequate learning even if there are not many examples to learn from. These weights were computed as n_samples / (n_classes × n_samples_per_class), providing a significant penalty for misclassifying rare attack types.

The learning rate of 0.05 was found to provide a gradual convergence and be less prone to overfitting, as proved by the validation and training loss reports. A maximum tree depth of 6 was also found to improve performance, probably due to the limited amount of trees forcing a much more reliable generalisation and way to prevent overfitting.

3.4.2 Deep Learning model based on FT-Transformer

A tabular DL transformer-based model was built in Pytorch using as a baseline TabTransformer(Huang *et al.*, 2020) and FT-Transformer (Gorishniy *et al.*, 2021). The pure original TabTransformer implementation diid not allow generalisation on both continuous data and categorical data so some modification had to be done, reason for why the

implementation does not use both “TabTransformer” and “FTTransformer” as names but it is referred across the text just as a “Transformer-based” model.

The architecture includes 3 main components. First, the input features are projected into a 32-dimensional embedding space using a linear transformation, providing sequences of feature embeddings. Then, the embeddings are processed using a stack of four transformer encoder layers, which in turn use multi-head self- attention and feed-forward sublayers for allowing the capture of contextual relationships between features.
Last, the output of the transformer is flattened and concatenated with the input features. The combined representation is passed through an MLP classification head.

Training made use of the Adam optimiser. Class imbalance was addressed by using weighted cross-entropy loss with class weights computed in the same way as the XGBoost baseline. Early stopping with a patience of 10 was applied. The model was trained with a batch size of 256. Training was set to a max of 50 epochs, and an early stopping with patience equal to 10 epochs based on F1-Score was used. Class-weighted cross-entropy loss was also set allowing penalties for minority class misclassification in a very similar way to XGBoost to allow a more accurate comparison. Training time was very low for both models, averaging at 45 seconds on a P100 GPU on Kaggle. Epoch 30-35 for the transformer-based model was enough for the model to finally converge.

The python code used included the selection of two libraries:
One of them is scikit-learn used for preprocessing, feature selection, evaluation. Pytorch was instead used for the transformer-based model. XGBoost was implemented by using the official Python wrappers which provides also a scikit-learn compatible interface.

3.5 Model Evaluation

The primary evaluation metric used was the macro-averaged F1 score. It consists of assigning equal weight to all ten classes regardless of support, making it robust to any class imbalance, very common to find in NIDS datasets, including LyCoS-Unicas-2018. In addition to the macro-averaged F1 score, per-class F1 score, recall, and per-class precision are reported alongside general accuracy. Accuracy is explicitly ignored as misleading because of the class distribution. Since benign traffic comprises around 73% of the total records, even a low-performing model predicting only benign traffic would still achieve 73% of accuracy.

Precision-recall curves, confusion matrices and per-class F1 are reported as supplementary evaluation. For completeness, per-class accuracy, general accuracy and weighted average are reported, although, as mentioned earlier, they are misleading.

3.6 Model Reproducibility

Training was performed by using 30 GB of RAM, a P100 GPU and an unknown CPU on Kaggle, a ML and DL website providing a notebook environment to both train, evaluate and deploy models. The notebook was then downloaded and adapted to run both on Kaggle and locally.

The username and paths inside the notebook for the dataset and model artefacts must be changed for the model to work. If run locally, artefacts must be extracted to the right folders and the model can both be trained and run in inference. The notebook can both load existing artefacts for fast evaluation, and it can be used to retrain a model from scratch.

A random seed of 42 was used across all relevant operations to ensure reproducibility. The complete pipeline which starts from dataset loading and goes through model training and evaluation, storage and deployment can be run cell by cell in chronological order to recreate all the experiments.

Software dependencies included:
- Python 3.10.13
- scikit-learn 1.3.2 (preprocessing, feature selection, metrics)
- XGBoost 2.0.3 (gradient boosting implementation)
- PyTorch 2.1.2 (deep learning framework)
- pandas 2.1.4 (data manipulation)
- NumPy 1.26.2 (numerical operations)

Chapter 4 Results

Chapter 4 contains all the results gathered during experimentation. It covers all the results from preprocessing, feature selection results, model performance during training and model performance on test set. Metrics used include model-by-model classification reports, their respective confusion matrix, and lastly a precision-recall curve.

Results are organized into three sections: feature selection outcomes, XGBoost final classification performance and the Transformer-based model performance.
All experiments were performed on a stratified sample of 100,00 instances which was split into 70/15/15 training, validation and test split.
Everything was optimised for macro average F1 score to avoid bias and enforce equal representation across all the classes present in the dataset. A confusion matrix and precision-recall curves were also used to measure performance and check for errors in prediction.

4.1 Feature selection

Permutation-based Recursive Feature Elimination with Cross-Validation (RFECV) was used to identify the best feature subset to use. 56 features out of the original 77 were selected from the LyCoS-Unicas-IDS2018 dataset. Feature selection used a Random Forest estimator with 5-fold stratified cross-validation using macro-averaged F1 score as optimization metric. The selected 56 features included characteristics belonging to the network flow such as destination port, forward and backward packet rates, TCP flags, bulk transfer metrics and inter-arrival time statistics.

- dst_port — primary discriminator for protocol-specific attacks
- fwd_pkt_per_s and bwd_pkt_per_s — packet rate characteristics
- iat_mean, iat_std, iat_max, iat_min — inter-arrival time patterns
- fwd_iat_mean, bwd_iat_mean — directional timing statistics
- flag_SYN, flag_fin, flag_rst, flag_ack, flag_psh — TCP connection behavior
- fwd_bulk_bytes_mean, bwd_bulk_bytes_mean — bulk transfer patterns
- active_mean, idle_mean — connection activity characteristics

The Zero-variance columns, flag_urg, fwd_flag_urg, and bwd_flag_urg were automatically removed by RFECV as mentioned earlier.

Permutation-based importance was used instead of the default impurity-based importance to avoid bias toward the high-cardinality features and the majority classes, as stated by Strobl et. al (2007) and Janitza et al. (2013). Using permutation-based feature importance ensures that minority classes receive equal weighting during selection critical for NIDS, where rare classes must be detected reliably.

Impurity-based feature selection, as mentioned earlier, retained 35 features against the 56 of permutation-based feature selection.

The 21 additional features retained by permutation-based feature selection are the ones contributing, even if in smaller amounts, in the retention of rare classes that impurity-based feature selection would otherwise discard. Impurity-based detection and its effects on model performance was not analysed as out of scope. The goal was to analyse the performance of the two models using the same reproducible input rather than analysing the performance of different feature selection techniques and their impact on performance.

An additional final note is that permutation-based feature selection was highly slower compared to impurity-selection.

However, it was accepted as trade-off for the better minority class retention. The slow runtime execution time affected only the feature selection phase and the models are not affected at all.
Also, after the 56 features are identified by the feature selection algorithm, they can be serialized in files to avoid the need for recomputing.

4.2 XGBoost

4.2.1 Deep Learning model based on FT-Transformer

XGBoost achieved good performance overall. Out of 14,997 instances present in the test set, the macro-averaged score reached 90.28%. The metric reveals balanced performance across several attack types while also highlighting problems in specific minority classes.

The following metrics were included in the classification report.

- Weighted F1: 98.93%
- Matthews Correlation Coefficient (MCC): 0.9770
- AUC Macro: 0.9994
- Macro Average Precision: 0.9191

- Macro Average Recall: 0.9322

XGBoost converged after 233 iterations using early stopping based on validation multi-class log loss. Final training log loss was 0.0177, while validation was 0.0184. The numbers being numerically closer indicate no overfitting being observable.

XGBoost does not require GPU acceleration and runs entirely on CPU. This means more power efficiency, especially when using histogram-based tree construction.

4.2.2 Per-class performance analysis

Performance of XGBoost across attacks varied significantly. Out of the used 10 classes, the following 7 achieved an F1-Score of 0.95 or higher as shown in Figure A-7. Benign class was the most performant class with only 15 false positives.

Class	Precision	Recall	F1 Score	Support
Benign	100.00%	99.86%	**99.93%**	10,956
Bot	90.60%	100.00%	**95.07%**	106
DDoS HOIC	99.75%	100.00%	**99.87%**	1,177
DDoS LOIC-HTTP	100.00%	100.00%	**100.00%**	317
DoS GoldenEye	96.67%	100.00%	**98.31%**	29
DoS Hulk	100.00%	99.95%	**99.97%**	1,975
DoS Slowloris	91.67%	100.00%	**95.65%**	11
SSH-Patator	100.00%	100.00%	**100.00%**	101

Table-B1

4.2.3 Problem Classes: Slowhttptest and FTP-Patator

Out of the 10 classes, two of them exhibited degraded performance due to bidirectional misclassification.

DoS Slowhttptest: 116 test instances, precision 45.53%, recall 45.53 % F1 Score 61.88%

FTP-Patator: 209 isntances, precision 94.94%, recall 35.89%, F1 Score 52.08%

This bidirectional confusion between the mentioned classes accounted for 91.4% of all the model errors. The pattern is asymmetric having most of FTP-Patator instances misclassified as Slowhttptest(64%) but only very few instances misclassified as FTP-Patator(3%).

Zero cross contamination occurred between the two classes and the other 8. The isolation between these classes allows the deployment of a tiered strategy in production where high-confidence classes can trigger completely automated responses while the poor performant classes can trigger alarms to human analysts for analysis.

4.3 Deep Learning

The Transformer-based model achieved a macro F1 score of 89.54% on the test set of 14,997 instances classifying correctly 14,864 of them. While results suggest the viability of attention-based architectures for NIDS the Transformer underperformed compared to XGBoost score of 90.28%. Supporting metrics included:

Weighted F1	99.07%
Matthews Correlation Coefficient (MCC)	0.9800

AUC Macro	0.9993
Macro Average Precision	0.8827
Macro Average Recall	0.9208

Table B-2

Training characteristics:

Training epochs**:**	14 (early stopping at epoch 14 with patience=10)
Best validation F1	0.8897 (epoch 4)
Test F1	0.8954 (slight degradation from validation peak)
Trainable parameters	281,930
Batch size	256
Embedding dimension:	32 (4 transformer layers, 4 attention heads per layer)

Table B-3

As XGBoost, performance on FTP-Patator and Slowhttptest was low indicating confusion between the two classes.

4.3.2 Per-class performance analysis

The Transformer displayed sufficient performance across classes, with seven classes achieving F1 > 0.95, but notable degradation for minority classes as shown in Figure A-13.

Class	Precision	Recall	F1 Score	XGBoost F1	Δ
Benign	100.00%	99.74%	**99.87%**	99.93%	-0.06pp
Bot	88.33%	100.00%	**93.81%**	95.07%	**-1.26pp**
DDoS HOIC	99.75%	100.00%	**99.87%**	99.87%	0.00pp
DDoS LOIC-HTTP	99.06%	100.00%	**99.53%**	100.00%	-0.47pp
DoS GoldenEye	96.67%	100.00%	**98.31%**	98.31%	0.00pp
DoS Hulk	100.00%	99.95%	**99.97%**	99.97%	0.00pp
DoS Slowloris	73.33%	100.00%	**84.62%**	95.65%	**-11.03pp**
SSH-Patator	97.12%	100.00%	**98.54%**	100.00%	-1.46pp

The two out of 10 problematic classes were further examined in both general performance of the model, by using the F1 score and confusion matrix for misclassification count.

DoS Slowhttptest (116 instances):

- F1: 43.24% (XGBoost: 61.88%)
- Confusion: 53 correctly classified, 63 misclassified as FTP-Patator

FTP-Patator (209 instances):

- F1: 77.68% (XGBoost: 52.08%)
- Confusion: 162 correctly classified, 47 misclassified as Slowhttptest

The transformer-based model showcased an opposite confusion bias where FTP-Patator was confused as Slowhttptest more often than Slowhttptest being confused as FTP-Patator.

4.3.3 Comparative Analysis Summary

XGBoost achieved a better Macro F1 score(90.28% against 89.54%), the primary evaluation metric, which indicates a more balanced detection against all classes. For usage in production environments, prioritization across all attack types is important and macro F1 score indicates XGBoost is more suitable for deployment.

This conclusion is valid for medium-sized datasets such as the one used for this research. Tree-based and deep learning models tend to operate at similar performance when datasets used contain more than 500,000 instances, compared to the 100,000 used for this experiment.

Model ranking in this study is not guaranteed to be retained once conditions such as dataset instance increase present.

However, with the conditions present in this study, the results are consistent across the reported metrics with XGBoost having a higher Macro F1 score and comparable results in per-class performance.

Both models have high performance, and the same faults in aspects such as misclassification observable especially in the precision-recall curve in Figure A-10.
While XGBoost has a higher score as can be seen in Figure A-11 the difference is close to 2%. This difference is calculated in a medium sized dataset which, considering Grinsztajn et al. (2022) research, means a bigger dataset will present a smaller difference between the two models making the gap negligible.

Chapter 5 Discussion

This chapter interprets the results obtained in Chapter 4. This contextualises the findings with existing literature on ML-based NIDS. Five key aspects were identified:

- Feature Selection
- XGBoost and Deep Learning comparison
- Implications and causes of Slowhttptest and FTP-Patator misclassification
- Usability in production environments
- Limitations of the study

5.1 XGBoost and Deep Learning Comparison

The 0.74% macro F1 score difference of XGboost over the transformer-based model (90.28% vs 89.54) confirms the findings of Grinsztajn et al. (2022), who demonstrated superior performance of tree-based models compared to DL models on medium-sized datasets (less than 500,000 instances).

The usage of the LyCoS-Unicas-IDS2018 dataset, proved decisive in the use of handling mechanisms for its class imbalance.
XGBoost sample weighting applies corrections at each split, allowing class separation of the minority class during construction of the tree. In contrast, the transformer-based model uses loss-level weighting, which applies a global penalty that propagates through multiple gradient descent steps to influence learned representations.

Architectural differences are observable, especially in minority class handling. XGBoost achieved high F1 scores for each class except for Slowhttptest (61.88%) and FTP-Patator (52.08%). The transformer-based model had mostly lower performance on all classes except for FTP-Patator, where the model achieved 77.68%.

While with large datasets with over 500,000 instances, tree-based algorithms achieve parity with DL algorithms, in this case, the dataset used contained 100,000 instances. Results align with findings from Gorishniy et al. (2021), where tree-based algorithms perform better on tabular data.

Although in the current research, both models trained relatively fast due to their small size, XGBoost still displays more computational efficiency than the transformer model. XGBoost was trained using only the CPU, while the transformer-based model used the GPU.

The absence of advantages in using Transformer-based DL algorithms suggests that the usage of tree-based algorithms is still preferred over DL models.

The trasnformer's self attention mechansism, designed to be able to capture long-range dependencies across sequential data(Waswani et al., 2017) could not provide any advantage for the unordered network flow data. The learned attention weights were unable to find any interaction pattern between the features useful enough to outperform XGBoost tree-based feature splits.

However, this does not imply that transformer-based models are unusable in NIDS. It only implies that performance on datasets of this size there is no significant advantage over tree-based models.
The usage of DL, such as transformer-based models, is more suitable when the data contains temporal sequences rather than, such as the case of this study, aggregated tabular data absent of any temporal pattern.
This is a complementary reason for why XGBoost is recommended for this setup and for why even by performing more aggressive hyperparameter tuning there is still a high probability of results not change regarding model ranking.

5.2 Persistent misclassification of FTP-Patator and DoS Slowhttptest classes

The bidirectional misclassification between DoS Slowhttptest and FTP-Patator (F1 61.88% and 52.08%) was the direct cause of 138 out of 151 misclassifications performed by XGBoost. However, there was zero cross-contamination present on the remaining classes and benign traffic. The pattern in question suggests a flow-level feature limitation rather than a model architecture issue.

5.2.1 Root Cause: Flow-Level Behavioural Similarity

The attacks exhibit very similar statistical characteristics as described in the Table 1.1 At the flow aggregation level, both attacks occupy the same statistical feature space by manifesting low-rate, small-payload and long-duration flows.

Table 1. 1

Aspects	Slowhttptest	FTP-Patator
Packet rate	less than 10 packetsper second (slow, persistent connections exhausting server resources)	less than 10 packets per second (brute force attempts with inter-attempt delays)
Packet size distribution	less than 100 bytes/packet (minimal HTTP headers, incomplete requests)	less than100 bytes/packet (FTP authentication commands: USER, PASS, failed responses)

Flow duration	more than 60 seconds (long-lived connections maintaining partial HTTP requests)	more than 60 seconds (iterative authentication attempts across sequential connection cycles)
Protocol behaviour at the flow level	Incomplete HTTP requests	Repeated FTP authentication failures, manifesting as low-rate command/response exchanges

5.2.2 Asymmetric Confusion Pattern and Feature Importance Analysis

The confusion demonstrates pronounced directionality: XGBoost misclassifies 64.1% of the instances from FTP-Patator as Slowhttptest, but only 3.4% of Slowhttptest instances as FTP-Patator. This asymmetry indicates that FTP-Patator's behavioural signature is a statistical subset of slowhttptest's signature space.

When examining permutation-based feature importance, the top discriminative features include:

- dst_port — FTP operates on ports 20/21, Slowhttptest targets port 80/443
- fwd_iat_mean, bwd_iat_mean — Inter-arrival time patterns differ slightly
- flag_SYN, flag_fin ratio — Connection lifecycle characteristics

These features can not be enough for class separaton as FTP can operate on other ports (FTP over HTTP proxies, high-port FTP services) and Slowhttptest can target any HTTP-capable service, not just port 80/443. Timing variance depends on the attacker's tooling configuration.

XGBoost's FTP-Patator classification rate of 36% suggests the model can successfully use port-based and timing features against a subset of instances, but the features are insufficient for consistent differentiation across the full class distribution.

5.2.3 Production Implications: Tiered Deployment Strategy

The isolation property, which means the zero cross-contamination between the two misclassified classes, Slowhttptest FTP-Patator, and the other classes, means that the following tiered strategy can be deployed:

- **Tier 1 (Automated Response): 8 classes with F1 ≥ 0.95**

DDoS HOIC, LOIC-HTTP, GoldenEye, Hulk (F1 ≥ 0.98)
Bot, Slowloris, SSH-Patator (F1 ≥ 0.95)
Benign (F1 0.9993, 0.14% FPR)
Action: Blocking, rate limiting, or traffic redirection

- **Tier 2 (Human Review Required): 2 classes with F1 < 0.65**

Slowhttptest (F1 0.6188) and FTP-Patator (F1 0.5208)
Action: Flag for security analysis review

- **Tier 0 (Baseline Monitoring): Benign traffic**

F1 0.9993, FPR 0.14% (15 false positives per 10,956 instances)

Action: Monitor for false positive rate escalation

This strategy can be effective in handling different threats without sacrificing operational feasibility.

5.3 Implications for Autonomous Firewall Design

Despite achieving a high 90.28% macro F1 score, several barriers prevent immediate production deployment.

5.3.1 Feature Extraction Latency

The LycosStand feature extraction pipeline has to be used for all network flows to generate a required 56 input vector for classification. This introduces variable latency which also depends on flow duration.

Flow competition, when all the features can be computed, can range from a couple of milliseconds, for example in the cases of DNS queries or web requests up to several minutes in cases such as file transfers or persistent connections.

For real-time intrusion detection this kind of latency is extremely limiting.

Also, before the flow is completed the model has no data usable for making predictions signifying a complete halt of the system for an unacceptable amount of time.

5.3.2 False positive Volume and Analysis Burden

Even if low (0.14%), the false positive rate observed on the test set amounts to over 1,022 daily false positives. This threshold is far from being sustainable for analyst workload.
An analyst can only process a limited number of alerts per day.

The resulted backlog would grow faster than the analyst team can handle making the system unsuitable for production.

5.3.3 Stability over time

A serious concern is the unstable nature of network traffic.

For each new application, protocol or attack tools, there will be a difference in the traffic patterns in network data that reaches the IDS.

A model trained on data from today might not be as performant when time passes.

In the case of this research, where evaluation was performed offline on a dataset, addressing this problem was out of scope.

However, in a real production environment, it is crucial to have the model monitored and retrained when needed before performance degrades enough to become hazardous for the network.

One way to do so might be to retest the model on a portion of production network data put aside from production analysed traffic, and check for performance degradation. If it is present, then the model has to be retrained with newer data.

Performance degradation in this scenario means a mesurable and singificant drop in per-class F1 score. While a small drop could indicate a low impact random variation in the sample from production, a consistent drop across all the classes or exaggerated drop in the benign class would be signal of models requiring retraining. Exact specifics for retraining are variable and depends on the specific production environment. This means that no information or statement can be given as this study is limited to using only an offline dataset with no test on real network data and workload.

5.4 Limitations of the study

The first limitation involves data sampling. The sampled data used for training amounts to 100,000 records, which comprise only 0.7% of the full

dataset. Even with stratification involved many classes had too few samples for effective convergence. Additionally, rare class removal also eliminated the ability to generalise to specific classes.

The experiment was also conducted on a single dataset. Cross-dataset generalisation was not tested. Literature consistently reports performance degradation on cross-dataset experimentation, suggesting that models used for these experiments will probably perform worse if tested on other datasets.

Train/test splitting is randomly sampled from the entire dataset rather than temporal separation. This creates the risk of data leakage if attack instances span multiple flows into both training and testing sets. While stratification preserves distribution, temporal evaluation better simulates production scenarios where models detect future attacks based on historical training.

Also, the architecture used is only a few in number. For feature selection, only RFECV with impurity-based and permutation-based were tested.

Only a single DL architecture was chosen based on FT-Transformer and TabTransformer. XGBoost is the only ML algorithm used. Many others could have been added and compared.

Additionally, while Macro F1 score allows to be more effective as performance metric than aggregated accuracy, it has the limitation of treating all classes equally. Treating all attack classes equally, is not realistic in a production environment as each attack has a different impact and brings different risks. For example, a bot infection is not equivalent in either impact or risk to a single DDoS HOIC attack flow.

While no suggestion can be given on how to address this limitation, this is a serious limitation to be overcome in order to have a production ready IDS.

Furthermore, another limitation involves the transformer-based model architecture and its configuration. Exhaustive tuning of parameters and configuration might have given different results. While scores support XGBoost performance as more efficient, there is the possibility of transformer-based architectures be more performant if using specific configurations.

The same can be applied to XGBoost. Defaults found in literature were used in hyperparameter tuning rather than performing an exaustive hyperparameter search.

The comparison of the models was focused more on general performance, rather than attempting to reach maximum capability for each of them. However, even if out of scope, a more accurate and extended hyperparameter search could lead to significative changes regarding performance.
Further hyperparameter search also must be carefully conducted.
This research ensured that input data for both models and their relative configuration were equal or similar allowing equal and fair comparison.
More hyperparameter tuning means danger of tuning one side more than the other bringing to unequal and unfair result during comparison.
This means that if further research is conducted on this aspect, the researcher must ensure retuning of the models is performed together and carefully not to bring bias in performance analysis.

Chapter 6 Conclusion and Future Work

This research evaluated the application of ML and DL algorithms on NIDS, trying to address the severe imbalance found generally in NIDS datasets, including LyCoS-Unicas-IDS2018, the choice for this dissertation. A comparison of two algorithms was performed, with XGBoost, a gradient boosting-based algorithm, and on the other, a Transformer-based DL algorithm.

The study addresses directly research questions regarding the usage of ML/DL in NIDS in particular, its viability, performance and production deployment.

6.1 Summary of Research Questions and Results

RQ1: Can ML and DL be used effectively in the NIDS domain?

Based on the results of the research, ML and DL can be used effectively in the NIDS domain, but with important clarifications. While both XGBoost and the Transformer based model achieved very good results overall in eight classes out of 10 with scores exceeding 95%, discrimination between similar classes, specifically FTP-Patator and Slowhttptest, which share the same statistical characteristics, proved to be a real limitation towards the applicability of the algorithms in a production environment.

RQ2: What are the performances of ML compared to DL?
XGBoost outperformed the Transformer-based DL model in this medium-sized dataset scenario. XGBoost achieved 90.28% macro F1 against 89.54% from the Transformer-based model.

The performance gap was especially high in minority classes such as: Bot, Slowloris, and Slowhttptest.

The only feature where the Transformer-based model achieved higher performance compared to XGBoost was on the FTP-Patator class, with

77.68% against 52.08%. This advantage seems to be class-specific not providing any systematic general advantage.

The findings seem to align with Grinsztajn et al. (2022) who claimed tree-based models outperform DL on medium-sized datasets. XGBoost's tree-level sample weighting seems to have been superior to loss-level weighting used by the transformer-based instance for addressing class imbalance.

Also, the computational efficiency of XGBoost is higher than that of the transformer-based counterpart. By using only, the CPU and having a shorter, even if only negligibly different, training time, it possesses a higher efficiency compared to the Transformer-based counterpart, which requires a GPU.

RQ3: Can both ML and DL be used effectively in a production-ready environment?

Both can be used partially, but not without the following clarifications:

- While 8 classes support automated response with high confidence, 2 classes still require manual intervention by means of security analysis
- While real-time inference can be achieved by the models themselves, captured traffic still requires feature extraction, increasing latency
- Cross-dataset generalisation is unknown: without knowing how well the model performs on other datasets, there is no guarantee of the performance on real-world data.

Having a tier development strategy, with automated response for high-confidence attacks while routing poor performing to an analysis team, is the safest and most robust option for deploying the models in production.

This means that an organisation deploying the proposed models would need to automate response the 8 high confidence classes and instead putting into an analyst queue both FTP-Patator and Slowhttptest for manual review.
It should also accept the latency involved for feature extraction from the raw traffic.

Even with these limitations, the model could still be of use, but it should be noted that complete automation cannot be fully reached and remains needed a certain amount of human interaction.
Both models are trained on a dataset which itself is limited to network data with no application-layer information.
Regardless of the model chosen, both inherit the same limitation which can be addressed only by changing feature extraction process.

6.2 Research Contributions

Three primary contributions to NIDS research are made by this research as follows.

- Direct comparison of XGBoost, an ML gradient boosting algorithm, to a Transformer-based DL algorithm sharing the same evaluation metrics (in particular, Macro F1).
- Validation of Grinsztajn et al. (2022) findings regarding XGBoost and other tree-based algorithms being more efficient in medium-sized datasets -with less than 500,000 instances (100,000 in this study).
- Bidirectional confusion between 2 of the 10 classes with zero-cross contamination.

6.3 Future Work

Cross-dataset generalisation is a necessary test to evaluate how the model behaves on other datasets from the one used for training. This eliminates dataset-specific patterns during evaluation. If the model performs poorly, it means it has learned dataset-specific patterns and not really generalised over the training data.
This is mandatory if the model is to be used on a production system, as this would mean a high risk of poor performance over real-world data.

Temporal split evaluation, temporal sequence extraction and protocol-derived feature extraction from raw PCAP files should be attempted to simulate accurately production environment and attempt to better discriminate slowhttptest and FTP-Patator.

An ensemble of the two used algorithms could be performed by leveraging each one's complementary error patterns.

Automated rule generation was not performed but is needed in a production environment. This can be done by developing SHAP-based rule extraction, which translates into usable firewall rules.

Adding application-layer features would also highly be beneficial.
As previously discussed, flow-level features produced by the LycosStand tool lack enough information to detect all classes as it has been the case in this study for FTP-Patator and Slowhttptest.
Having application-layer features such as packet content, or protocol-specific descriptors could allow more performant predictions and ability to better separate classes.

One practical way to achieve this, would be to extend LycosStand feature set and perform HTTP request header parsing and protocol-specific flow in general.

This is the primary method to improve F1 score including for FTP-Patator and Slowhttptest.

This method of adding new features during feature extraction for increase in performance should be attempted first to avoid unnecessary and costly redesign of the pipeline such as changing of the model or changing to the dataset.

List of References

Abdelaziz, M.T. et al. (2025) 'Enhancing network threat detection with random forest-based NIDS and permutation feature importance', Journal of Network and Systems Management, 33(1), p. 2. Available at: https://doi.org/10.1007/s10922-024-09874-0.

Ali, A.H. et al. (2024) 'Unveiling machine learning strategies and considerations in intrusion detection systems: a comprehensive survey', Frontiers in Computer Science, 6, p. 1387354. Available at: https://doi.org/10.3389/fcomp.2024.1387354.

Awad, M. and Fraihat, S. (2023) 'Recursive feature elimination with cross-validation with decision tree: Feature selection method for machine learning-based intrusion detection systems', Journal of Sensor and Actuator Networks, 12(5), p. 67. Available at: https://doi.org/10.3390/jsan12050067.

Blagus, R. and Lusa, L. (2013) 'SMOTE for high-dimensional class-imbalanced data', BMC Bioinformatics, 14(1), p. 106. Available at: https://doi.org/10.1186/1471-2105-14-106.

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Buitinck, L. et al. (2013) 'API design for machine learning software: Experiences from the scikit-learn project', in ECML PKDD Workshop: Languages for Data Mining and Machine Learning, pp. 108–122.

Cantone, M., Marrocco, C. and Bria, A. (2024) 'On the cross-dataset generalization of machine learning for network intrusion detection', IEEE Access, 12, pp. 144489–144508. Available at: https://doi.org/10.1109/ACCESS.2024.3472907.

Catalano, C. et al. (2022) 'Deceiving AI-based malware detection through polymorphic attacks', Computers in Industry, 143, p. 103751. Available at: https://doi.org/10.1016/j.compind.2022.103751.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', Journal of Artificial Intelligence Research, 16, pp. 321–357. Available at: https://doi.org/10.1613/jair.953.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. Available at: https://doi.org/10.1145/2939672.2939785.

Chen, W. et al. (2024) 'A survey on imbalanced learning: Latest research, applications and future directions', Artificial Intelligence Review, 57(6), p. 137. Available at: https://doi.org/10.1007/s10462-024-10759-6.

Chicco, D. and Jurman, G. (2020) 'The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation', BMC Genomics, 21(1), p. 6. Available at: https://doi.org/10.1186/s12864-019-6413-7.

Chinnasamy, P. et al. (2025) 'A systematic review of deep learning-based network intrusion detection systems', ICT Express. Available at: https://doi.org/10.1016/j.icte.2025.01.005.

Fang, R. et al. (no date) 'LLM Agents can Autonomously Exploit One-day Vulnerabilities'. Available at: https://arxiv.org/abs/2404.08144.

Goldschmidt, P. and Chudá, D. (2025) 'Network intrusion datasets: A survey, limitations, and recommendations', Computers & Security, 156, p. 104510. Available at: https://doi.org/10.1016/j.cose.2025.104510.

Gorishniy, Y., Rubachev, I., Khrulkov, V. and Babenko, A. (2021) 'Revisiting deep learning models for tabular data', in Advances in Neural

Information Processing Systems, 34, pp. 18932–18943. Available at: https://arxiv.org/abs/2106.11959.

Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022) 'Why do tree-based models still outperform deep learning on tabular data?', in Advances in Neural Information Processing Systems, 35, pp. 507–520. Available at: https://arxiv.org/abs/2207.08815.

Guyon, I. and Elisseeff, A. (2003) 'An introduction to variable and feature selection', Journal of Machine Learning Research, 3, pp. 1157–1182.

Guyon, I., Weston, J., Barnhill, S. and Vapnik, V. (2002) 'Gene selection for cancer classification using support vector machines', Machine Learning, 46(1-3), pp. 389–422.

Huang, X., Khetan, A., Cvitkovic, M. and Karnin, Z. (2020) 'TabTransformer: Tabular data modeling using contextual embeddings', arXiv preprint. Available at: https://arxiv.org/abs/2012.06678.

Janitza, S., Strobl, C. and Boulesteix, A.L. (2013) 'An AUC-based permutation variable importance measure for random forests', BMC Bioinformatics, 14(1), p. 119. Available at: https://doi.org/10.1186/1471-2105-14-119.

Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: Formulation, detection, and avoidance', ACM Transactions on Knowledge Discovery from Data, 6(4), pp. 1–21. Available at: https://doi.org/10.1145/2382577.2382579.

Khraisat, A. et al. (2019) 'Survey of intrusion detection systems: Techniques, datasets and challenges', Cybersecurity, 2(1), p. 20. Available at: https://doi.org/10.1186/s42400-019-0038-7.

Kohavi, R. (1995) 'A study of cross-validation and bootstrap for accuracy estimation and model selection', in Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1143.

Kothamali, P.R. and Banik, S. (2022) 'Limitations of signature-based threat detection', International Journal of Computer Applications, 13(01).

Leevy, J.L., Khoshgoftaar, T.M., Bauder, R.A. and Seliya, N. (2018) 'A survey on addressing high-class imbalance in big data', Journal of Big Data, 5(1), p. 42. Available at: https://doi.org/10.1186/s40537-018-0151-6.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', in Advances in Neural Information Processing Systems, 30. Available at: https://arxiv.org/abs/1705.07874.

Meinshausen, N. and Bühlmann, P. (2010) 'Stability selection', Journal of the Royal Statistical Society: Series B, 72(4), pp. 417–473. Available at: https://doi.org/10.1111/j.1467-9868.2010.00740.x.

Oh, S. (2023) 'Feature selection with permutation-based variable importance for imbalanced classification', arXiv preprint. Available at: https://arxiv.org/abs/2312.10573.

Paszke, A. et al. (2019) 'PyTorch: An imperative style, high-performance deep learning library', in Advances in Neural Information Processing Systems, 32. Available at: https://arxiv.org/abs/1912.01703.

Pinheiro, J.M.H. et al. (2025) 'The impact of feature scaling in machine learning: Effects on regression and classification tasks', IEEE Access, 13, pp. 199903–199931. Available at: https://doi.org/10.1109/ACCESS.2025.3635541.

Riyanto, S. et al. (2023) 'Comparative analysis using various performance metrics in imbalanced data for multi-class text classification', International Journal of Advanced Computer Science and Applications, 14(6). Available at: https://doi.org/10.14569/IJACSA.2023.01406116.

Rosay, A., Carlier, F., Cheval, E. and Leroux, P. (2022) 'From CIC-IDS-2017 to LYCOS-IDS-2017: A corrected dataset for better performance',

IEEE Access, 10, pp. 95821–95849. Available at: https://doi.org/10.1109/ACCESS.2022.3205067.

Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', PLOS ONE, 10(3), e0118432. Available at: https://doi.org/10.1371/journal.pone.0118432.

Scornet, E. (2021) 'Trees, forests, and impurity-based variable importance', arXiv preprint. Available at: https://arxiv.org/abs/2001.04295.

Sharafaldin, I., Lashkari, A.H. and Ghorbani, A.A. (2018) 'Toward generating a new intrusion detection dataset and intrusion traffic characterisation', in Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). Funchal, Madeira, Portugal: SCITEPRESS, pp. 108–116. Available at: https://doi.org/10.5220/0006639801080116.

Shin, J. et al. (2021) 'Effects of class imbalance on resampling and ensemble learning for improved prediction of cyanobacteria blooms', Ecological Informatics, 61, p. 101202. Available at: https://doi.org/10.1016/j.ecoinf.2020.101202.

Sommer, R. and Paxson, V. (2010) 'Outside the closed world: On using machine learning for network intrusion detection', in Proceedings of the IEEE Symposium on Security and Privacy. Oakland, CA, USA: IEEE, pp. 305–316. Available at: https://doi.org/10.1109/SP.2010.25.

Strobl, C., Boulesteix, A.L., Zeileis, A. and Hothorn, T. (2007) 'Bias in random forest variable importance measures: Illustrations, sources and a solution', BMC Bioinformatics, 8(1), p. 25. Available at: https://doi.org/10.1186/1471-2105-8-25.

Tripathi, N. and Hubballi, N. (2018) 'Application layer denial-of-service attacks and defense mechanisms: A survey', ACM Computing Surveys, 51(4), pp. 1–55. Available at: https://doi.org/10.1145/3243773.

Vaswani, A. et al. (2017) 'Attention is all you need', in Advances in Neural Information Processing Systems, 30, pp. 5998–6008. Available at: https://arxiv.org/abs/1706.03762.

Wang, K. and Stolfo, S.J. (2004) 'Anomalous payload-based network intrusion detection', in Jonsson, E., Valdes, A. and Almgren, M. (eds.) Recent Advances in Intrusion Detection. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 203–222. Available at: https://doi.org/10.1007/978-3-540-30143-1_11.

Zhang, Y., Muniyandi, R.C. and Qamar, F. (2025) 'A review of deep learning applications in intrusion detection systems: Overcoming challenges in spatiotemporal feature extraction and data imbalance', Applied Sciences, 15(3), p. 1552. Available at: https://doi.org/10.3390/app15031552.

Bibliography

Howard, J., and Gugger, S. 2020. *Deep Learning for Coders with fastai and PyTorch: AI Applications Without a PhD*. O'Reilly Media.